Course: Artificial Intelligence and Machine Learning Code: 20CS51I

WEEK - 10: DEEP LEARNING

- **Introduction to TensorFlow**
- **What is TensorFlow?**
- **Use Cases/Applications of TensorFlow:**
- **TensorFlow Ecosystem:**
- **Architecture of TensorFlow**
- **TensorFlow's Basic Programming Elements**

Session No. 3

Introduction to TensorFlow:

TensorFlow is an open-source software library. **TensorFlow** was originally developed by researchers and engineers working on the Google Brain Team within Google's Machine Intelligence research organization for the purposes of conducting machine learning and deep neural networks research, but the system is general enough to be applicable in a wide variety of other domains as well! Let us first try to understand what the word **TensorFlow** actually mean! **TensorFlow** is basically a software library for numerical computation using **data flow graphs** where:

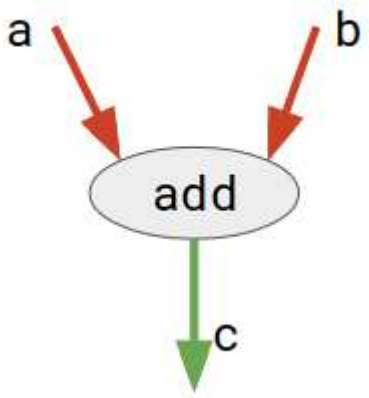
- **nodes** in the graph represent mathematical operations.
- **edges** in the graph represent the multidimensional data arrays (called **tensors**) communicated between them. (Please note that **tensor** is the central unit of data in TensorFlow).

What is TensorFlow?

TensorFlow is a popular framework of **machine learning** and **deep learning**. It is **a free** and **open-source** library which is released on **9 November 2015** and developed by **Google Brain Team**. It is entirely based on Python programming language and use for numerical computation and data flow, which makes machine learning faster and easier.

TensorFlow can train and run the deep neural networks for image recognition, handwritten digit classification, recurrent neural network, **word embedding**, **natural language processing**, video detection, and many more. TensorFlow is run on multiple **CPU**s or **GPU**s and also mobile operating systems.

Consider the diagram given below:



Here, **add** is a node which represents addition operation. **a** and **b** are input tensors and **c** is the resultant tensor. This flexible architecture allows you to deploy computation to one or more CPUs or GPUs in a desktop, server, or mobile device with a single API!

TensorFlow APIs

TensorFlow provides multiple APIs (Application Programming Interfaces). These can be classified into 2 major categories:

1. Low level API:
    - complete programming control
    - recommended for machine learning researchers
    - provides fine levels of control over the models
    - **TensorFlow Core** is the low level API of TensorFlow.
2. High level API:
    - built on top of **TensorFlow Core**
    - easier to learn and use than **TensorFlow Core**

- make repetitive tasks easier and more consistent between different users
- **tf.contrib.learn** is an example of a high level API.

In this article, we first discuss the basics of **TensorFlow Core** and then explore the higher level API, **tf.contrib.learn**.

TensorFlow Core

1. Installing TensorFlow

An easy to follow guide for **TensorFlow** installation is available here: Installing TensorFlow. Once installed, you can ensure a successful installation by running this command in python interpreter:

import tensorflow as tf

2. The Computational Graph

Any **TensorFlow Core** program can be divided into two discrete sections:

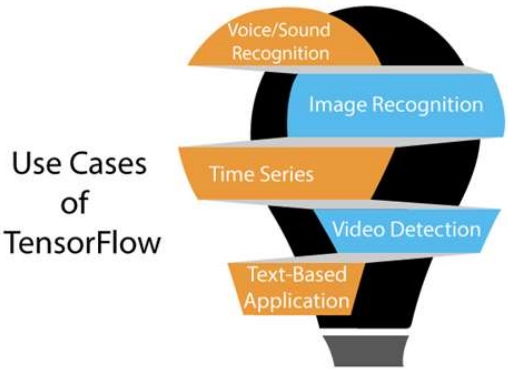
- Building the computational graph.A **computational graph** is nothing but a series of TensorFlow operations arranged into a graph of nodes.
- Running the computational graph.To actually evaluate the nodes, we must run the computational graph within a **session**. A session encapsulates the control and state of the TensorFlow runtime.

Why tensorflow?

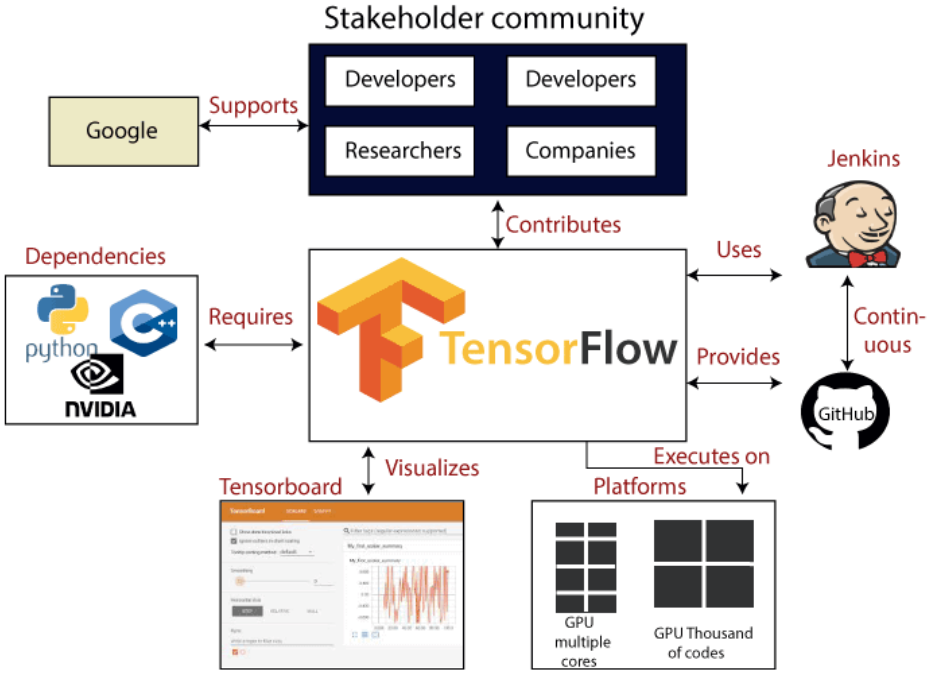
TensorFlow is the better library for all because it is accessible to everyone. TensorFlow library integrates different **API** to create a scale deep learning architecture like **CNN (Convolutional Neural Network)** or **RNN (Recurrent Neural Network)**.

TensorFlow is based on graph computation; it can allow the developer to create the construction of the neural network with Tensorboard. This tool helps debug our program. It runs on CPU (Central Processing Unit) and GPU (Graphical Processing Unit).

Use Cases/Applications of TensorFlow:



TensorFlow provides amazing functionalities and services when compared to other popular deep learning frameworks. TensorFlow is used to create a large-scale **neural network** with many layers.



It is mainly used for deep learning or machine learning problems such as **Classification**, **Perception**, **Understanding**, **Discovering Prediction**, and **Creation**.

1. Voice/Sound Recognition

Voice and sound recognition applications are the most-known use cases of deep-learning. If the neural networks have proper input data feed, neural networks are capable of understanding audio signals.

For example:

Voice recognition is used in the Internet of Things, automotive, security, and UX/UI.

Sentiment Analysis is mostly used in customer relationship management **(CRM)**.

2. Image Recognition

Image recognition is the first application that made deep learning and machine learning popular. Telecom, Social Media, and handset manufacturers mostly use image recognition. It is also used for face recognition, image search, motion detection, machine vision, and photo clustering.

For example, image recognition is used to recognize and identify people and objects in from of images. Image recognition is used to understand the context and content of any image.

For object recognition, TensorFlow helps to classify and identify arbitrary objects within larger images.

This is also used in engineering application to identify shape for odelling purpose (**3d** reconstruction from **2d** image) and by Facebook for photo tagging.

3. Time Series

Deep learning is using Time Series algorithms for examining the time series data to extract meaningful statistics. For example, it has used the time series to predict the stock market.

A recommendation is the most common use case for Time Series. **Amazon**, **Google**, **Facebook**, and **Netflix** are using deep learning for the suggestion. So, the deep learning algorithm is used to

analyze customer activity and compare it to millions of other users to determine what the customer may like to purchase or watch.

For example, it can be used to recommend us TV shows or movies that people like based on TV shows or movies we already watched.

4. Video Detection

The deep learning algorithm is used for video detection. It is used for motion detection, real-time threat detection in gaming, security, airports, and UI/UX field.

For example, NASA is developing a deep learning network for object clustering of asteroids and orbit classification. So, it can classify and predict NEOs (**Near Earth Objects**).

5. Text-Based Applications

Text-based application is also a popular deep learning algorithm. Sentimental analysis, social media, threat detection, and fraud detection, are the example of Text-based applications.

For example, Google Translate supports over 100 languages.

Some **companies** who are *currently using TensorFlow* are Google, AirBnb, eBay, Intel, DropBox, Deep Mind, Airbus, CEVA, Snapchat, SAP, Uber, Twitter, Coca-Cola, and IBM.

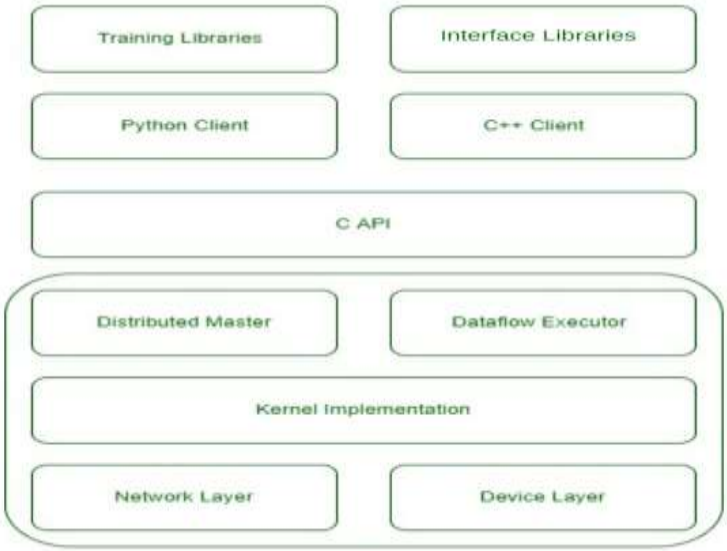
TensorFlow Ecosystem:

To Build and fine-tune models with the TensorFlow ecosystem

Explore an entire ecosystem built on the Core framework that streamlines model construction, training, and export. TensorFlow supports distributed training, immediate model iteration and easy debugging with Keras , and much more. Tools like Model Analysis and TensorBoard help you track development and improvement through your model's lifecycle.
To help you get started, find collections of pre-trained models at TensorFlow Hub from Google and the community, or implementations of state-of-the art research models in the Model Garden . These libraries of high-level components allow you to take powerful models, and fine-tune them on new data or customize them to perform new tasks.

Visit: TensorFlow

Architecture of TensorFlow



High-level Architecture

Architecture of TensorFlow

- The first layer of TensorFlow consists of the device layer and the network layer. The device layer contains the implementation to communicate to the various devices like GPU, CPU, TPU in the operating system where TensorFlow will run. Whereas the network layer has

implementations to communicate with different machines using different networking protocols in the Distributable Trainable setting.

- The second layer of TensorFlow contains kernel implementations for applications mostly used in machine learning.
- The third layer of TensorFlow consists of distributed master and dataflow executors. Distributed Master has the ability to distribute workloads to different devices on the system. Whereas data flow executor performs the data flow graph optimally.
- The next layer exposes all the functionalities in the form of API which is implemented in C language. C language is chosen because it is fast, reliable, and can run on any operating system.
- The fifth layer provides support for Python and C++ clients

TensorFlow's Basic Programming Elements

TensorFlow allows us to assign data to three kinds of data elements: constants, variables, and placeholders.

Let's take a closer look at what each of these data components represents.

1. Constants

Constants, as the name implies, are parameters with fixed values. A constant in TensorFlow is defined using the command tf.constant(). Constant values cannot be altered during computation.

Here's an example:

```
c = tf.constant(2.0,tf.float32)

d = tf.constant(3.0)

Print (c,d)
```

2. Variables

Variables allow new parameters to be added to the graph. The tf.variable() command defines a variable that must be initialized before the graph can be run in a session.

Here's an example:

```
Y = tf.Variable([.4],dtype=tf.float32)

a = tf.Variable([-.4],dtype=tf.float32)
```

b = tf.placeholder(tf.float32)

linear_model = Y*b+a

3. Placeholders

Data can be fed into a model from the outside using placeholders. It later permits the assignment of values. A placeholder is defined using the command tf.placeholder().

Here's an example:

c = tf.placeholder(tf.float32)

d = c*2

result = sess.run(d,feed_out={c:3.0})

The placeholder is mostly used to input data into a model. Data from the outside are fed into a graph via a variable name (in the above example, the variable name is fed out). Following that, we specify how we want to feed the data to the model while executing the session.

Example of a session:

The graph is executed by launching a session. The TensorFlow runtime is used to evaluate the nodes in the graph. The command sess = tf. Session() begins a session.

Example:

x = tf.constant(3.0)

y = tf.constant(4.0)

z = x+y

sess = tf.Session() #Launching Session

print(sess.run(z)) #Evaluating the Tensor z

There are three nodes in the preceding example: x, y, and z. The mathematical operation is performed at node 'z,' and the result is received afterward. When you start a session and run the node z, the nodes x and y are produced first. The addition operation will then occur at node z. As a result, we will get the answer'7.